

VQE senza termine DM — trimero a catena aperta

Note di lavoro — Tesi triennale, Samuele Schiavi

Relatore: Prof. Alessandro Chiesa

Introduzione

Questo documento raccoglie i file della fase VQE **senza** termine DM per il trimero a catena aperta — mirror diretto, per $N = 3$, di quanto fatto per il dimer (`vqe_dimer.py`) e per l'anello (documento gemello). Un ansatz PMA con branching e blocco RBS (produzione principale), lo stesso con blocco W , un ansatz PMA senza branching, due notebook "spiegato" (guide interattive) e un notebook di confronto sistematico.

Scope. Tutti i file qui descritti evitano deliberatamente il termine DM. Per la catena, il VQE con DM è ancora completamente da fare — le scelte qui descritte non dipendono da come verrà affrontato in futuro. Per i moduli `.py`, oltre a descrizione/verifiche/conclusione, sono elencate anche le funzioni interne e come si usa il file.

`vqe_trimer_chain.py` + `vqe_trimer_chain.png`

Mirror diretto di `vqe_trimer_ring.py`, costruito da zero in questa sessione: ansatz HA (9 parametri) e PMA con RBS e branching sui soli due bond fisici (1,2) e (2,3). Per $J > 0$ il fondamentale a basso campo è sempre il blocco B (mai serve scegliere fra B e C come nell'anello): $2K + 3$ parametri, 5 a $K = 1$ — contro i 6 dell'anello, un blocco RBS in meno.

Verifiche. Self-test a precisione macchina (1.9×10^{-15}), script principale completo eseguito: $\mathcal{F} = 1$ esatto su tutti i 10 punti, per entrambi gli ansatz. Figura ispezionata visivamente.

Conclusione. Modulo di produzione pronto, stesso standard di rigore dell'anello.

Funzioni interne.

- `make_ha Ansatz(reps)` — l'ansatz hardware-efficient generico, stesso ruolo della versione dell'anello.
- `rbs_block(qc, phi, q0, q1)` — il blocco RBS, identico a quello dell'anello.
- `_select_target_block(J, b)` — sceglie (`block`, M) del fondamentale: sempre B sotto b_c per $J > 0$ (nessun confronto E_B vs E_C necessario, a differenza dell'anello), A sopra.
- `_prepare_initial_state(qc, block, M)` — banale (solo X) per A estremo, generico (`prepare_state`) per B — qui B non è mai banale (a differenza del blocco a basso campo tipico dell'anello).
- `make_pma Ansatz(J, b, K)` — l'ansatz PMA: stato iniziale + K layer di RBS sui *due* bond fisici + R_y indipendenti finali, $2K + 3$ parametri.
- `_energy(params, Ansatz, hamiltonian, estimator)` — la funzione obiettivo per `scipy.optimize.minimize`.
- `run_vqe(J, b, Ansatz_type, reps, K, n_restarts, seed)` — VQE completo a un punto (nessun `dm_mode/D`: qui lo scope esclude il DM fin dalla firma della funzione, non solo dal suo uso).

- `vqe_sweep(b_values, J, ansatz_type, reps, K, n_restarts, seed, verbose)` — chiama `run_vqe` su una griglia di b .
- `_self_test()` — verifica l'ansatz a parametri nulli contro lo stato di Kambe atteso.
- `_plot_row, plot_grid` — stessa funzione della versione dell'anello, per la figura a 2×4 pannelli.

Come si usa. Come libreria: `from vqe_trimer_chain import run_vqe, vqe_sweep, ....`
 Come script standalone: `python3 vqe_trimer_chain.py` esegue il self-test, poi lo sweep completo per HA e PMA e salva `vqe_trimer_chain.png/.pdf`. Dipende da `trimer_chain_exact.py` nella stessa cartella.

`vqe_trimer_chain_W.py + vqe_trimer_chain_W.png`

Mirror di `vqe_trimer_ring_W.py`: gate W , branching, $2K$ parametri (2 a $K = 1$, contro i 3 dell'anello).

Verifiche. Self-test a precisione macchina, script principale eseguito: $\mathcal{F} = 1$ esatto ovunque, con errore energetico esattamente nullo in più punti sopra b_c .

Conclusione. Completa, insieme al file precedente, la coppia RBS/ W già disponibile per l'anello.

Funzioni interne. Stessa struttura di `vqe_trimer_chain.py`, con queste differenze:

- `w_block(qc, theta, q0, q1)` — il blocco W (CNOT- R_y -CNOT) al posto di `rbs_block`.
- `make_pma_ansatz(J, b, K)` — solo K blocchi W sui due bond, $2K$ parametri, nessun layer finale di R_y .
- `make_pma_ansatz_forced(branch, K)` — ramo iniziale fissato indipendentemente da b ; per il ramo "basso" usa `prepare_state` sullo stato di Kambe ($B, -\frac{1}{2}$) (diversamente dall'anello, dove il ramo "basso" forzato usa la preparazione banale del singoletto).
- `run_vqe_forced(J, b, branch, K, n_restarts, seed)` — il VQE con il ramo forzato.
- Le altre funzioni (`make_ha_ansatz`, `_select_target_block`, `_prepare_initial_state`, `_self_test`, `_energy`, `run_vqe`, `vqe_sweep`, `_plot_row`, `plot_grid`) hanno lo stesso ruolo della versione RBS.

Come si usa. Come libreria: `from vqe_trimer_chain_W import run_vqe, vqe_sweep, ....` A differenza di `vqe_trimer_chain.py`, in questa sessione è stato eseguito tramite uno script esterno equivalente (stesso schema self-test + sweep + plot). Dipende da `trimer_chain_exact.py`.

`vqe_trimer_chain_nobbranch.py`

Mirror di `vqe_trimer_ring_nobbranch.py`: stato iniziale fisso, RBS ripetuto ad ogni ciclo sui due bond fisici, $5K$ parametri (contro i $6K$ dell'anello).

Verifiche e scoperta. Trovato e corretto un bug reale durante la costruzione: il conteggio dei parametri per ciclo era sbagliato (4 invece di 5), causava un `IndexError` immediato. Dopo la correzione, verificato un risultato non presunto per analogia: a $b = 1.0$ (sotto b_c), $K = 1$ (5 parametri) resta vicino al tetto strutturale $\approx 5/6$ già noto per la famiglia "esatta" (RBS fatto una sola volta), ma $K = 2$ (10 parametri, RBS ripetuto due volte) lo **supera**, raggiungendo $\mathcal{F} = 1$ esatto — coerente col pattern già visto per l'anello, dove la famiglia ciclica a $K = 2$ recupera l'esatto sotto DM dove $K = 1$ resta bloccato.

Conclusione. Ripetere il blocco RBS (non solo aggiungere R_y) è ciò che apre nuovo spazio raggiungibile — un'osservazione strutturale, non specifica del DM, che vale anche nel regime $D = 0$ qui documentato.

Funzioni interne.

- `rbs_block(qc, phi, q0, q1)` — stesso blocco RBS già validato in `vqe_trimer_chain.py`.
- `make_ha_ansatz(reps)` — l'ansatz hardware-efficient, non usato dal resto del modulo ma incluso per uniformità con gli altri file.
- `pma_2q_trimer_nobbranch(K)` — l'ansatz senza branching: X fisso su un qubit, poi K cicli di [RBS sui *due* bond + R_y indipendenti sui tre qubit], $5K$ parametri (era scritto erroneamente come $4K$ prima della correzione).
- `_energy(params, ansatz, hamiltonian, estimator)` — stessa funzione obiettivo degli altri moduli.
- `run_vqe(J, b, K, n_restarts, seed)` — VQE a un punto, interfaccia analoga agli altri moduli ma senza `ansatz_type/reps`.
- `vqe_sweep(b_values, J, K, n_restarts, seed, verbose)` — chiama `run_vqe` su una griglia di b .

Come si usa. Come libreria: `from vqe_trimer_chain_nobbranch import run_vqe, vqe_sweep`. Nessun blocco `__main__`, nessuna funzione di self-test dedicata — la verifica più diretta è chiamare `run_vqe` su un punto qualsiasi e controllare `fidelity`. Dipende da `trimer_chain_exact.py`.

`vqe_trimer_chain_spiegato.ipynb`

Mirror di `vqe_trimer_ring_spiegato.ipynb`, con un adattamento fisico non banale: il blocco a basso campo della catena (B) non è banale come il blocco C usato nel punto di lavoro standard dell'anello, quindi la sezione di derivazione a mano (X, H, CX, X) non si applica — sostituita con una verifica numerica di `prepare_state`.

Verifiche e correzione. Eseguito per intero: trovato un problema reale, non un bug di codice ma di **presentazione** — due celle (VQE a un punto singolo, e confronto su griglia) mostravano $\mathcal{F} = 0.826612$ invece di 1.000000 , non per un limite strutturale ma per un minimo locale specifico del seed di default (42). Diagnostico con più restart ($R = 8$ risolve) e con seed diversi (qualunque seed $\neq 42$ risolve già a $R = 4$). Corretto cambiando il seed esplicito nelle due celle; rieseguito, ora $\mathcal{F} = 1.000000$ esatto ovunque.

Conclusione. Un esempio concreto, verificato sul campo, della distinzione fra "limite di espressività" e "artefatto di ottimizzazione" che il progetto tiene ferma fin dal dimero: qui il rischio di scambiare l'uno per l'altro era reale e si è materializzato, non solo teorico.

`vqe_trimer_chain_W_spiegato.ipynb`

Mirror di `vqe_trimer_ring_W_spiegato.ipynb`, con lo stesso adattamento della sezione di derivazione del notebook precedente (blocco B non banale).

Verifiche. Eseguito per intero, zero errori: overlap di preparazione = 1.000000 , HA e PMA entrambi $\mathcal{F} = 1$ esatto sulla griglia ridotta.

Conclusione generale

La catena ha ora una famiglia di file per la fase VQE senza DM parallela a quella dell'anello (produzione RBS, produzione W , produzione senza branching, due guide interattive), con lo stesso livello di verifica end-to-end. Due punti di attenzione emersi durante la costruzione, entrambi reali e non solo teorici:

- **Il seed di default può ingannare:** il problema trovato in `vqe_trimer_chain_spiegato.ipynb` mostra che un valore di fidelity "sospetto" (vicino a un tetto già noto per un'altra famiglia) va sempre verificato con più restart o seed diversi prima di essere interpretato come limite strutturale.
- **Ripetere un blocco non è come aggiungere parametri a un blocco fisso:** la scoperta su `vqe_trimer_chain_nobranh.py` ($K=2$ supera il tetto di $K=1$) generalizza un principio già visto per l'anello sotto DM, qui confermato anche a $D = 0$.

Nota. `confronto_ansatz_entangler_trimer_catena.ipynb` non è descritto in questo documento: pur essendo attualmente $D = 0$ -only, è lo stesso slot di notebook che per l'anello è poi diventato il confronto sistematico esteso al DM (`confronto_ansatz_entangler_trimer_anello.ipynb`). Verrà documentato, nella sua forma definitiva, insieme al resto della Fase 5 (VQE con DM) per la catena, non ancora affrontata.